

CS575
Woonki Kim.
kimwoon@oregonstate.edu
7 April 2024

Project #0 Simple OpenMP Experiment

1. Machine Used

- I used the engr flip2 server to run the code.

2. Performance Measurements

To measure the performance for thread 1, I have set `NUMT` to 1. Similarly, I have set `NUMT` to 4 to measure the performance for thread 4. The variable `NUMT` represents the number of threads, and I utilized `omp_get_wtime()` to measure the execution time. Time measurement is initiated by setting `time_0` before entering the main computation loop and `time_1` after exiting the loop. The time consumed is then calculated by subtracting `time_0` from `time_1`. Finally, to evaluate the performance(MegaMults/Sec), the formula $\frac{\text{SIZE}}{\text{time}_1 - \text{time}_0} \times 10^6$ is used, where `SIZE` represents the size of the array.

- For 1 thread, Peak Performance = 437.58 MegaMults/Sec
- For 4 threads, Peak Performance = 1240.74 MegaMults/Sec

Threads	Performance (MegaMults/Sec)
1	437.58
4	1240.74

3. One-Thread-to-Four-Threads Speedup

- $S = \frac{\text{Performance with four threads}}{\text{Performance with one thread}} = \frac{1240.74}{437.58} \approx 2.8343$

4.Reason Why One-Thread-to-Four-Threads Speedup Should Be Less Than 4.0

- The work in the inner loop might not be evenly distributed among the threads. If some iterations take significantly longer to execute than others, threads may finish their work at different times, leading to idle time for some threads and overall inefficient utilization of resources.
- When multiple threads are accessing shared resources concurrently, contention can occur, leading to delays and inefficiencies. This contention increases with the number of threads, and at a certain point, it can outweigh the benefits gained from parallelization, resulting in diminishing returns in speedup.

5.Parallel Fraction

- $Fp = \left(\frac{4}{3}\right) \times \left(1 - \frac{1}{2.8343}\right) = 0.86293333$